SkyJet Flight Recovery

Real-World Deployment Blueprint

What this is: The end-to-end plan for taking the current MVP from a laptop demo to a production airline system. It answers, concretely, what we need to *add* for **rebooking**, **refund**, and **waiting / waitlist** — how the system finds options (nearby + related flights), which data structures we use, what logic runs, how the database changes even after a flight is already booked, and which databases and external systems we will need.

Grounded in: the actual codebase (`skyjet-recovery/`). Companion to `architecture.md` (MVP as-built) and `features.md` (scope).

Challenge	22North Product Engineering Challenge 2026 — Challenge 1 (PS1)
Document type	Production / real-world deployment design
Scope	Rebooking · Refund · Waiting (waitlist) · Databases · Integrations
Date	3 July 2026
Status	Roadmap behind the MVP — the answer to “how would this actually work?”

Contents

(In Word: right-click → Update Field to populate page numbers.)

1. How to read this document

Every section is written as Today (MVP) → Production adds. The MVP is deliberately a single polished vertical slice: an in-memory store, mock APIs, same-route rebooking, and a refund that issues a reference number only. Nothing here contradicts that scope — this is the roadmap behind the “Future Work” slide, so every deferral in the demo has a real answer when a judge asks “how would this actually work?”

The one principle that carries over: the authority lives in the human + the database, never in a guess. Every state change is idempotent, race-safe, and audited — already true in the MVP, and only more important at production scale, because during a disruption **everyone acts at once**.

2. The gap at a glance

Concern	Today (MVP)	Production adds
Data store	In-memory Maps in store.ts, reset on cold start	PostgreSQL (OLTP) + Redis (cache/holds) + more (§3)
Inventory	One seatsAvailable integer per flight	Real seat inventory + airline PSS/GDS as system of record
Related flights	Linear scan, same origin→destination only	Indexed availability query + hot route cache; PSS availability API
Nearby options	None	Geospatial airport search + connection graph
Auth	PNR + surname (findBooking)	Real IdP / SSO + OTP; short-lived tokens
Refund	Eligibility → reference RF-…, no payment	Refund ledger + payment gateway; versioned rules for audit
Waitlist	Priority computed in-request, not stored	Durable waitlist + event-driven auto-promotion + timed holds
Notifications	Mock “alert that would be sent”	Queue (SQS/Kafka) + Twilio/WhatsApp/SES, outbox + DLQ
Policy answers	RAG wired to Gemini + Pinecone	Same pipeline; managed vector DB; ops-uploadable corpus

3. Databases & data stores we will need

The MVP runs on one in-memory store that already mirrors the production Postgres schema 1:1 (prisma/schema.prisma) — including the optimistic-concurrency version column and the idempotency-key unique constraint. Production spreads the load across purpose-built stores:

Store	Technology	What it holds	Why it exists
Core OLTP	PostgreSQL (multi-AZ, read replicas)	passenger, booking, flight, audit_event, idempotency, refund, waitlist, seat_hold	Transactional source of truth; row locking for seat writes
Cache / low-latency	Redis	Flight-status cache, route-availability (ZSET), seat holds (TTL), waitlist sorted-sets, geo index, rate limits	Disruptions are read spikes — keep hot reads off Postgres

Store	Technology	What it holds	Why it exists
Geo / search	PostGIS (GiST) or OpenSearch	Airport lat/long, metro codes, ground-transfer times; flight search	Answers “airports near X” without full scans
Vector DB	Pinecone (integrated) / pgvector	Policy-clause embeddings for the Disruption Assistant	Grounded, cited entitlement answers; text stays canonical
Event streaming	Kafka or SNS+SQS	Disruption events, rebook/refund sagas, notification fan-out, outbox, DLQ	Async, ordered, replayable side-effects
Object storage	S3	Boarding-pass PDFs, refund receipts, policy docs	Large binaries don’t belong in the OLTP DB
Warehouse	BigQuery / Snowflake	Historical audit + KPI facts (deflection, time-to-reaccommodate, refund-vs-rebook)	Impact tile + ops dashboards off the OLTP path
Observability	Datadog / OpenSearch / Prometheus	Logs, metrics, distributed traces	SLOs, saga debugging, PII-redacted audit

Systems of record we integrate with (not ours to own): the airline **PSS/GDS** (Amadeus Altéa, Sabre, or Navitaire) is the real owner of bookings, inventory, availability, and ticket reissue. In production our Postgres is effectively a **consistent cache + workflow layer** in front of the PSS — the seat decrement in §4.5 is ultimately a call into the PSS, wrapped in a saga.

4. Rebooking in the real world

The demo answers “can I move to another flight?” by scanning the seeded flights for the same route. Production has to (a) find related flights robustly, (b) find nearby-airport and connecting alternatives when the direct route is exhausted, (c) rank them, and (d) commit the change safely against real inventory.

4.1 Finding related flights (same origin → destination)

Today. store.alternativesFor(flight) linearly scans the flights Map and keeps those that are SCHEDULED, not DEPARTED, same origin/destination, seatsAvailable > 0, and departing at/after the original — then sorts by departure. Correct for a handful of seeded flights; O(n) per lookup.

Production.

- **Data structure:** a B-tree composite index (origin, destination, status, departure) on the flight table (already declared in schema.prisma), so the candidate set is a range scan, not a table scan. In front of it, a Redis sorted set per origin|destination|date (score = departure epoch) is the hot route-availability cache so a spike doesn’t hammer Postgres.
- **Source of truth:** availability actually lives in the PSS/GDS. We call its availability API for the OD pair over a time window (same-day → next 24–48h), cache the result in Redis with a short TTL, and treat our copy as a cache of theirs.
- **Logic:** query availability for the OD + date window → filter by cabin/fare class and seats > 0 → hand the candidate list to the scorer (§4.4). Recomputed live so a shown option is never stale.

4.2 Finding nearby flights (the geospatial part)

When the same route is sold out or cancelled, the next-best recovery is often a nearby airport — depart from a sister airport in the same metro, or land at a neighbouring city and add a short ground transfer. This is the part the MVP does not attempt, and it needs real spatial data plus a spatial index.

Data we need: an airport reference table — IATA/ICAO code, latitude/longitude, city, IATA metropolitan-area code (all London airports group under LON, all NYC under NYC), and typical ground-transfer time between neighbours. Sourced from OAG / Cirium (or OpenFlights for a prototype).

Data structure — a geospatial index. To answer “airports within R km of X” in O(log n) instead of scanning every airport, use one of:

- **Geohash** buckets — natively supported by Redis GEO (GEOADD / GEOSEARCH), which stores points in a sorted set keyed by geohash; ideal for the hot path.
- **R-tree / GiST index** in PostGIS (ST_DWithin) — best when the geo query joins other flight predicates in SQL.
- **k-d tree** — a good in-memory option if the airport set is loaded into a service.
- Plus a precomputed **metro grouping** hash map (cityCode → [airports]) for the common same-metro case, which needs no distance math at all.

Logic:

1. Build a candidate origin set = {original origin} ∪ {airports within R_o km} (e.g. R_o = 150 km) via the geo index, using the **Haversine / great-circle formula** for distance.

2. Build a candidate destination set the same way around the destination.
3. Search flights over the (nearby-origins × nearby-destinations) product.
4. In the scorer, **penalise** each option by the added ground-transfer time to/from the passenger's real airport, so a nearby-airport option only wins when it is meaningfully better (much earlier arrival).
5. Enforce a **maximum detour** threshold; beyond it, route to an agent rather than auto-offer.

4.3 Connecting / multi-leg reaccommodation — model the network as a graph

"Related" (one direct flight) and "nearby" (endpoint substitution) are both special cases of the general question: what is the earliest valid way to get this passenger to their destination? The clean way to express that is a graph.

- **Data structure — a directed flight graph** (time-expanded network): nodes are airports (or (airport, time) states); edges are scheduled flights weighted by arrival time / duration / cost; "connection" edges enforce a minimum-connection-time constraint at each hub.
- **Logic — shortest-path search** (Dijkstra / A* with arrival-time as the cost, or a time-expanded BFS) finds the earliest-arriving itinerary, naturally including 0-stop (§4.1), 1–2 stop options, and nearby-airport endpoints (§4.2) in one framework. Cap at 1–2 connections for self-service; deeper or interline itineraries escalate.
- **Why a graph:** it unifies direct, nearby, and connecting search, and it is exactly where partner/interline rebooking (a future phase) plugs in — as extra edges.

4.4 Scoring & the recommended "best option"

Today. scoreOption() starts every candidate at 100 and subtracts penalties (−30 next-day, −5/h later capped at −40, +10 within 2h of the original time-of-day), attaching a plain-English reason ("Same day · 3h later · direct"). The top-scored option the passenger can actually take is flagged recommended. This is our answer to American Airlines' tool being criticised as opaque.

Production adds more factors to the same transparent model — fare/cost difference, connection risk, on-time performance, the ground-transfer penalty from §4.2, and cabin/loyalty retention. To pull the best N from a large candidate set without fully sorting it, use a bounded max-heap / priority queue (top-K selection). The explainability contract stays: every option ships a human-readable reason.

Capacity rationing (already implemented). During IRROPS the alternative flights are not empty, so scarce seats are rationed by priority (priority.ts, rebooking-priority.ts): senior → business → child/infant → standard. Seats are held for still-unaccommodated higher-priority passengers; a lower-priority passenger who can't yet be seated is waitlisted — exactly the "waiting" flow in §6.

4.5 The rebooking WRITE path — "even if a flight is already booked, how do we change the DB?"

This is the transactional-integrity core. Because everyone rebooks at once during a disruption, the write path must be idempotent, race-safe, and atomic — the MVP already does this in-memory; production makes it a real DB transaction (and, ultimately, a PSS call).

1. Client sends POST /api/rebook with a client-generated **idempotency key** (implemented).
2. If that key was already seen → return the **stored** response, 200 idempotent-replay: true (implemented; a DB UNIQUE constraint enforces it in production).

3. **Revalidate** the chosen flightId is still in the freshly-computed options, else 409 — guards a stale tab / race (implemented).
4. BEGIN a database transaction.
5. **Optimistic lock:** UPDATE booking SET … WHERE ref = ? AND version = ?. If 0 rows updated, someone else changed the booking → 409, client retries on fresh state (the version column is already modelled).
6. **Decrement seat inventory atomically** on the new flight: UPDATE flight SET seatsAvailable = seatsAvailable − 1 WHERE id = ? AND seatsAvailable > 0 (conditional update = row-level lock; 0 rows → the flight just filled → fall through to the waitlist, §6). With a real seat map this flips a seat_inventory row HELD → BOOKED. In an airline, this is a call into the PSS/inventory system, wrapped in the saga.
7. **Release the old seat** back to inventory (seatsAvailable + 1 on the original flight / free the seat_hold).
8. Update the booking: status = REBOOKED, rebookedFlightId, chosen seat, version++, updatedAt.
9. **Append an audit / outbox row** (action REBOOK, before/after) in the same transaction — the transactional-outbox pattern, so the event can't be lost or double-sent.
10. Persist the idempotency key → response row.
11. **COMMIT.** An outbox worker then publishes the event to the queue → downstream: issue the new boarding pass, reroute bags, update loyalty, refresh DCS, send the notification.

The tables that change on a single rebook:

Table	Change
booking	status → REBOOKED, rebookedFlightId, seat, version++, updatedAt
flight / seat_inventory	new flight seat reserved (−1 / HELD→BOOKED); old flight seat released (+1)
audit_event / outbox	one append-only row (before/after, trace-id)
idempotency	one row: key → stored response

Concurrency note: the new-flight seat row is the contended hot row during IRROPS. Conditional updates + bounded retries + graceful failover into the priority waitlist mean a double-tap can never double-book and a race can never oversell — it degrades into a waitlist offer instead.

5. Refund in the real world

Today. evaluateEligibility() is a pure, cause-driven rules engine (weather/ATC/security = extraordinary → free rebook or full refund + duty-of-care, no cash compensation; technical/crew/ops = airline-controlled → the above plus tiered ₹5,000/₹7,500/₹10,000 cash), each field carrying a reason and a DGCA rule citation. POST /api/refund checks eligibility.refund.eligible, issues a reference RF-XXXXXX, sets status = REFUND_REQUESTED, and audits it. No payment integration — reference number only (a deliberate, stated constraint).

Production adds (payment intentionally still designed-not-built until later, but the surrounding machinery is real):

- **A versioned rules service.** The same cause-driven logic, but each decision is persisted with the exact rule version that produced it, so the airline can defend it to a regulator (DGCA claim window is 2 years). Thresholds/tiers move into a rules/config table (or the existing RAG policy corpus) so ops can update them without a deploy. EU261 tiers slot in alongside DGCA for international routes.
- **A refund ledger.** A refund table modelled as a state machine: REQUESTED → APPROVED → PROCESSING → PAID | FAILED — with amount, currency, method (original form of payment / voucher / eCredit), gateway transaction id, and timestamps. Double-entry style so finance can reconcile.
    - **Data structure:** an explicit state machine (states + allowed transitions) backed by the refund row, driven forward by queue events and gateway webhooks.
- **Payment/refund gateway (designed interface).** When enabled, PROCESSING calls the gateway to refund to the original form of payment; the call is idempotent by refund reference; settlement is async (DGCA: ~7–15 working days) and a webhook flips the row to PAID. The reference number we already issue is what lets the passenger track it — so the MVP's reference-only design is forward-compatible, not a dead end.
- **Voucher / eCredit alternative** — a stored-value account credited instantly; cheaper for the airline, faster for the passenger, and the refund-vs-voucher split becomes a tracked KPI.

DB changes on a refund request: booking.status = REFUND_REQUESTED + refundReference; a new refund row (REQUESTED); an audit_event/outbox row. Later, the gateway webhook advances the refund row to PAID (or FAILED → retry / escalate) and may re-open seat inventory for resale.

6. Waiting / waitlist in the real world

What "waiting" means here. During a disruption the alternative flights have only a few spare seats, so they are rationed by priority — and a passenger who can't be seated yet is waitlisted for that flight. In the MVP this is computed in-request by capacityFor(): it counts how many higher-priority passengers are still competing and marks the option available: false with the note "Seats held for N higher-priority passengers … you're on the waitlist for this flight." Correct and explainable — but stateless (nothing is stored, nothing happens when a seat later frees).

Production adds — this is the "what else can we do for waiting" answer:

- **A durable waitlist.** A waitlist table keyed by (flightId, bookingRef) with priorityRank, requestedAt, and state WAITING → OFFERED → CONFIRMED | EXPIRED.
- **Data structure — a priority queue.** Per flight, a binary min-heap ordered by (priorityRank ASC, requestedAt ASC) (senior → business → child/infant → standard, FIFO within a tier — matching computePriority). In Redis this is a sorted set per flight with score = priorityRank × 10^{13} + requestedAt, giving atomic "pop the most-deserving waiter."
- **Event-driven auto-promotion.** When a seat frees — a cancellation, a no-show, an inventory increase, or an aircraft up-gauge — an event fires → pop the head of that flight's waitlist → create a time-boxed seat hold (a seat_hold row with a TTL, e.g. 15 min) → notify the passenger ("a seat opened on SJ456 — tap to accept within 15 min"). On accept, run the normal rebook write path (§4.5); on expiry, release the hold and offer the next in line.
- **Standby** — an airport-day variant of the same queue (fly the next open seat at the gate).
- **Overbooking / oversell management** — track held vs confirmed vs physical capacity; if oversold, run denied-boarding compensation, which feeds straight back into the eligibility engine.
- **Proactive comms while waiting** — surface the passenger's position in queue ("you're #3"), an ETA, and the duty-of-care they're owed now (meals after 2h, hotel overnight) — reusing the goodwill / duty-of-care logic already in the model. Waiting should never be silent.

DB changes for the waitlist: insert/advance waitlist rows; create/expire seat_hold rows (TTL); audit/outbox events on every transition; on promotion, the standard rebook write path commits the seat.

7. External systems & integrations we will need

System	Example	Why we need it
PSS / GDS	Amadeus Altéa, Sabre, Navitaire	System of record for bookings, inventory, availability, ticket reissue
Availability / inventory	Airline inventory service	Real-time seat counts the rebook write path decrements
Departure control (DCS)	Amadeus / Sabre DCS	Boarding passes, check-in, live seat maps
Schedule + airport data	OAG, Cirium	Routes, times, airport lat/long, metro codes (powers §4.2 geo search)
Weather / ATC feeds	METAR/TAF, NOTAM, ATC flow	Auto-classify disruption cause (drives eligibility)
Payment / refund	Airline payment service + gateway	Actual disbursement (deferred build; interface designed in §5)

System	Example	Why we need it
Notifications	Twilio SMS, WhatsApp, SES/SendGrid, push	Proactive alerts + the deep-link into the flow
Identity	Auth0 / Cognito / airline SSO + OTP	Replace PNR+surname with real, revocable auth
Loyalty (FFP)	Frequent-flyer platform	Tier-aware handling, mileage re-accrual on rebooking
Baggage	Bag-tracking system	Reroute checked bags with the rebooked itinerary
Agent desktop / CRM	Salesforce / Zendesk	Receive the warm-handoff context pack for escalations

8. Non-functional, security & compliance for production

- **Scale (disruptions are traffic spikes).** Stateless services behind a load balancer with autoscale; cache flight status (Redis/CDN); async notifications via a queue; idempotent + optimistically-locked writes (all already in the MVP design); Postgres read replicas; circuit breakers and timeouts on PSS calls so a slow GDS can't cascade.
- **Security.** Real IdP/SSO + OTP; TLS everywhere; secrets in a managed vault; rate limiting (ratelimit.ts already present); least-privilege service roles.
- **Privacy & regulatory.** DPDP (India) + GDPR for international passengers: PII redaction in logs, data-retention windows, and explicit consent for notification channels. DGCA compliance record-keeping — persist every eligibility decision with its rule version for the 2-year claim window (the audit_event table is the seed of this).
- **Reliability.** Multi-AZ Postgres with PITR backups + DR; DLQ + retries on the notification / refund sagas; distributed tracing and SLOs; the append-only audit trail for governance and debuggability.

9. Phased rollout roadmap

Phase	Scope
0 — MVP (today)	In-memory store, mock APIs, same-route rebooking, refund = reference number, mock notifications, RAG-lite assistant. The polished demo slice.
1 — Real backbone	Postgres + Redis; read real PSS/GDS availability; the idempotent/race-safe write path against real inventory; notifications via queue + Twilio/WhatsApp; IdP + OTP.
2 — Recovery depth	Nearby-airport + connection graph search (§4.2–4.3); durable waitlist + auto-promotion (§6); refund ledger + payment gateway (§5); loyalty + baggage.
3 — Enterprise / scale	Predictive (pre-disruption) alerts; interline/partner rebooking as graph edges; event-driven microservices (Kafka + saga + outbox + DLQ); cost/OR-optimised reaccommodation; full managed RAG at corpus scale.

10. Summary — the three flows at a glance

Flow	How it finds options	Data structure	Core logic	DB changes	External systems
Rebooking	Indexed same-route query + geo search for nearby airports + graph shortest-path for connections	B-tree route index; Redis GEO / R-tree / k-d tree; time-expanded flight graph; top-K heap	Haversine radius → candidate OD product → Dijkstra/A* → transparent scoring; idempotent, locked, atomic seat decrement	booking, flight/seat_inventory (×2), audit/outbox, idempotency	PSS/GDS, availability, DCS, OAG/Cirium, weather
Refund	Cause-driven eligibility over the disrupted booking	Versioned rules table; refund state machine	Extraordinary vs airline-controlled → refund/comp/duty-of-care with rule citation; async settlement	booking (status + ref), new refund row, audit/outbox	Payment/refund gateway (deferred), rules/config store
Waiting / waitlist	Priority rationing when seats are scarce	Priority queue (heap) / Redis sorted-set per flight; seat_hold with TTL	Rank senior→business→child→standard (FIFO in tier); event-driven auto-promotion + timed hold + notify	waitlist rows, seat_hold (TTL), audit/outbox, then rebook write path	Notifications, inventory, DCS

Appendix — current code → production component

MVP file	Role today	Production evolution
src/lib/store.ts	In-memory Maps + alternativesFor	Postgres repositories + Redis caches; alternativesFor → indexed availability + geo/graph search
prisma/schema.prisma	Documented DB schema (mirrors the store)	Migrated Postgres schema + new refund, waitlist, seat_hold, seat_inventory tables
src/lib/eligibility.ts	Pure DGCA rules engine	Versioned rules service; decisions persisted with rule version
src/lib/service.ts	Scored options + idempotent rebook	Multi-factor scoring over graph candidates; write path becomes a DB transaction + PSS saga
priority.ts, rebooking-priority.ts	In-request priority + capacity	Backs the durable waitlist + auto-promotion queue
src/lib/rag/*	Gemini + Pinecone policy assistant	Same pipeline; managed vector DB + ops-uploadable corpus
src/lib/ratelimit.ts	Basic rate limiting	Edge/gateway rate limiting + WAF
AuditEntry / audit log	Every mutation recorded	Append-only audit + outbox → warehouse; DGCA compliance record

Compiled 3 July 2026 from the live skyjet-recovery/ codebase. Production-facing companion to architecture.md (MVP as-built) and features.md (scope).